

Motore Scacchistico

Simone Bastoni

6 luglio 2026

1 Aggiornamento codice

In ordine di rilevanza visiva le modifiche sono:

- Creazione di file : è stato aggiunto il file `partita.hpp/.cpp` per una gestione più pulita del codice inerente alle azioni di gioco.
- Rettifica codice : il codice é stato aggiornato secondo le linee guida di C++20. È stato sistemato l'incapsulamento delle funzioni in classi e struct e sono stati utilizzati algoritmi STL.
- Implementazione mosse : sono stati aggiunti il doppio passo del pedone e la presa di en passant (anche chiamata presa al varco) per adeguarsi alle regole moderne di gioco.
- Ottimizzazioni : sono state ottimizzate le funzioni di ricerca della mossa migliore per ridurre le tempistiche di gioco o per potersi spingere più in profondità nella ricerca senza penalizzare eccessivamente la fluidità.
- Simulazioni : sono state effettuate simulazioni di gioco con Stockfish per poter attribuire un punteggio ELO al mio motore.

Il codice completo è disponibile nella Repository GitHub del progetto.

2 Descrizione Progetto

Il progetto è volto all'implementazione di un motore scacchistico in C++ giocabile da terminale. È dotato di algoritmi per la generazione di mosse che permettono di giocare in modalità Umano vs Bot oppure anche in modalità spettatore assistendo a una partita Bot vs Bot. In aggiunta alla classica modalità Umano vs Umano.

3 Scelte Progettuali

Il motore scacchistico é diviso in sei parti:

- Generatore di mosse : contiene le funzioni e le strutture dati per la generazione delle mosse possibili (che poi verranno filtrate in struttura).
- Struttura : racchiude tutta la parte riguardante la gestione della scacchiera, il controllo delle mosse e la gestione dei turni. Contiene anche i controlli sulle condizioni di fine partita quali: triplice ripetizione, scacco matto e stallo (gli ultimi 2 solo in maniera indiretta).
- Bot : contiene gli algoritmi per la valutazione di scacchiera e mosse, quelli di scelta e gli algoritmi per l'efficientamento del codice.
- Partita : contiene tutte le funzioni di gestione della partita, la gestione della modalità, dei cronometri e delle condizioni di fine partita.
- Main : gestisce le risorse degli altri file e si occupa dell'inizializzazione del programma.
- Test : contiene i test implementati con Doctest per verificare la correttezza dei calcoli del codice.

Tra le scelte di sviluppo significative si notifica che la promozione del pedone è stata indirizzata univocamente alla Regina senza possibilità di scelta. Con l'aggiunta del doppio passo del pedone e la presa di *en passant*, l'unica regola esclusa dal programma è la regola delle 50 mosse (regola 5.2/e del regolamento FIDE) ma data la bassa incidenza statistica in cui questa regola viene applicata si è deciso di ignorarla; la sua assenza non compromette l'esperienza di gioco. Il tempo impostato per giocare è pari a 600 s (10 minuti). Inizialmente il cronometro di gara era stato impostato a 15 minuti a causa del procedimento inusuale di input della mossa, ma dopo diversi tentativi è stato deciso di rimettere una tempistica standard di 10 minuti.

3.1 Algoritmo MiniMax

È l'algoritmo più famoso di tutti gli scacchi ed è la base utilizzata per la parte Bot del mio motore. Si tratta di un algoritmo ricorsivo (al suo interno chiama se stesso) per la simulazione e la valutazione nel tempo della scacchiera. Per natura degli scacchi è possibile produrre simulazioni oggettive sull'esito di alcune mosse e quindi prevedere quale sia la mossa migliore per ogni situazione.

L'algoritmo genera un albero di mosse a crescita esponenziale. A ogni mossa l'algoritmo chiama una funzione per la valutazione statica della scacchiera che restituisce il valore della scacchiera in quel momento. Vengono svolte delle simulazioni su ogni possibile mossa per un numero determinato di turni. L'algoritmo restituisce la mossa che porta il Bot nella situazione migliore. La funzione di valutazione si basa sulla valutazione PST (Piece-square Table) ossia delle tabelle che assegnano un bonus o un malus (rispetto a un valore standard) a ogni pezzo sulla base della posizione nella scacchiera.

Vista la quantità enorme di mosse che genera MiniMax è necessario ottimizzare l'algoritmo per permettere di ottenere risultati in tempi ragionevoli. La

prima ottimizzazione che ho implementato prende il nome di "Potatura alfa beta", alfa é il valore da massimizzare e beta é il valore da minimizzare: se in qualche punto dell'albero delle mosse, per il turno di alfa (che massimizza), si ottiene $\alpha < \beta$, tutto il ramo viene scartato perchè significa che la mossa analizzata porta l'avversario in una posizione più vantaggiosa di una posizione già analizzata e quindi é inutile continuare. Questo é possibile perché alfa viene inizializzata con un valore di $-\infty$ e riaggiornata ogni volta che viene trovato un valore migliore. Per beta la situazione é speculare ad alfa ma il ragionamento é lo stesso.

Un'ulteriore ottimizzazione apportata al MiniMax riguarda l'ordinamento delle mosse. Nel caso di ordinamento ideale la complessità computazionale subirebbe una riduzione da $\mathcal{O}(b^d)$ a $\mathcal{O}(b^{d/2})$, con un valore medio di $\mathcal{O}(b^{3d/4})$ (dove b indica il numero di mosse legali per turno, mediamente 35, mentre d indica la profondità di esplorazione). Al lato pratico con una ottimizzazione ideale, si otterrebbe una riduzione delle mosse per nodo da circa 35 a circa 6, permettendo a parità di tempo di analizzare le mosse a profondità doppia (la profondità indica fino a quanti turni nel futuro si spinge la simulazione); tutto questo "solo" analizzando prima le mosse che statisticamente sono migliori (come catture, arroccchi e promozioni) favorendo così i tagli Alfa-Beta. Per fare ciò ho implementato una funzione di valutazione delle mosse che assegna un punteggio alla mossa sulla base di alcune sue caratteristiche intrinseche (il punto di arrivo, se promuovo, se catturo qualcosa e con cosa, etc...) e una funzione che le ordina sulla base di questi punteggi. Per l'ordinamento delle mosse é stata sfruttata la tecnica MCV - LVA, ossia vengono favorite le catture di pezzi grandi (regine, torri, etc...) con pezzi piccoli (pedoni, cavalli, etc...), tutto in funzione all'incremento delle potature dei nodi. Dopo l'ultima revisione ho aggiunto il riordinamento delle mosse per ogni nodo esplorato sulla base del punteggio della mossa precedente (che statisticamente se una mossa é buona anche le successive lo saranno).

Le ultime ottimizzazioni sono per il controllo dinamico della profondità di ricerca e ricerca di quiete. La ricerca di quiete evita il fenomeno dell'effetto orizzonte tale per cui il bot pensa di aver trovato una mossa ottima (magari la cattura della regina avversaria con la propria) senza però aver valutato la mossa successiva perché si trovava già al limite di profondità impostato (nella mossa successiva l'avversario gli mangiava la regina annullando completamente il vantaggio). La ricerca di quiete fa continuare l'esplorazione fintanto che ci sono catture disponibili. Il controllo dinamico della profondità invece modifica la profondità massima di ricerca della mossa (quanti turni nel futuro simula l'algoritmo) per poter ottimizzare al meglio la performance sulla base del momento della partita in cui ci si trova (nel finale si può andare più a fondo poiché ci sono meno pezzi e l'albero delle mosse é più stretto).

3.2 Altri algoritmi

Per il controllo sulla triplice ripetizione é stato necessario implementare una chiave della scacchiera. Si tratta di una funzione che genera una stringa che identifica la scacchiera in ogni sua posizione e a ogni mossa controlla che non

esistano piú altre due situazioni simili sennó scarta la mossa (lo stallo da punteggio zero, quindi chi é in vantaggio lo evita).

I controlli sullo scacco matto sono "indiretti" dentro struttura perché sono situazioni che si verificano quando non ci sono piú mosse legali da poter fare. Il controllo sulle mosse legali avviene su Struttura ma é nel file Partita che viene deciso se l'assenza di mosse legali porta a uno stallo o a uno scacco matto.

La ricerca delle mosse legali avviene filtrando dalle tabelle precompilate di `generatore_di_mosse` (genera ogni mossa possibile per ogni pezzo in ogni posizione) e controlla prima i limiti geometrici (non ci siano altri pezzi che blocchino) e poi attraverso la funzione `re_sotto_scacco` controlla se la mossa mette in pericolo o meno il proprio re.

4 Uso di costruttori non introdotti a lezione

Tutte le basi necessarie allo sviluppo del programma seguono gli argomenti trattati a lezione. È stata però necessaria una fase di studio individuale per imparare a sviluppare e strutturare algoritmi di ricerca e analisi dati piú avanzati.

5 Istruzioni per l'esecuzione

Il codice sorgente è suddiviso nei seguenti file: `main.cpp`, `partita.cpp`, `struttura.cpp`, `bot.cpp` e `generatore_di_mosse.cpp`; sono presenti i seguenti file header: `struttura.hpp`, `bot.hpp`, `partita.hpp` e `generatore_di_mosse.hpp`. Per la compilazione é necessario l'utilizzo di **C++20**. È inoltre presente anche il file `test.cpp` che contiene dei test (implementati attraverso l'uso di DOCTEST) per verificare che il programma sia quanto piú possibile esente da bug dovuto a errori di calcolo. È presente anche il file `CMakeLists.txt` contenente le informazioni per la compilazione. Se si utilizzasse un sistema di compilazione differente da CMake é vivamente consigliato l'utilizzo dell'opzione di ottimizzazione del compilatore (nel caso di g++ l'utilizzo del comando `-O3`) per rendere il motore scacchistico il piú performante possibile.

6 Parametri in Input e in Output

Una volta lanciato il programma in modalità Release ci si troverá davanti a una sezione di menú dove viene richiesto digitare 1 o 2 per selezionare la modalità di gioco:

- 1 Umano vs Bot
- 2 Bot vs Bot (spettatore)
- 3 Umano vs Umano

si scriva sotto di fianco a "Scelta" la modalità in cui si vuole giocare. Deve essere inserito unicamente il numero della modalità e nessun altro carattere altrimenti il programma lancerà il messaggio "MODALITÀ NON VALIDA!" e si chiuderà.

Se si sceglie di giocare in modalità spettatore [2] non è necessario inserire nessun altro parametro di input, si assiste alla partita, al cui termine uscirà il verdetto e il programma terminerà in autonomia. Durante lo svolgimento, a ogni turno, verranno proiettati a schermo le mosse rispettive dei bot, il tempo utilizzato per l'elaborazione e il tempo mancante allo scadere del cronometro; la scacchiera si aggiornerà in autonomia a ogni turno.

Se viene scelta la modalità [1] (Umano vs Bot), l'umano giocherà con i pezzi bianchi (i pezzi pieni) mentre il Bot utilizzerà i pezzi neri (quelli vuoti). Sotto la Scacchiera contenente la posizione corrente sopraggiungerà il messaggio "inserisci mossa"; in questo caso la forma corretta di input è di inserire la casella di partenza, poi uno spazio e successivamente la casella di arrivo (es. per muovere in avanti di 1 il pedone posizionato davanti al re digitare "e2 e3"). Nel caso si sbagli a inserire la mossa o si inserisca una mossa non valida (es. il re è sotto scacco e si muove un altro pezzo che non andrà a coprirlo), sopraggiungerà il messaggio "Mossa impossibile, riprova!" seguito da "Tentativi rimasti (es. 4/5)". È stato inserito il limite di 5 mosse impossibili prima di lanciare il messaggio "TROPPI ERRORI CONSECUTIVI" e chiudere il programma. Se la mossa inserita è una mossa possibile, uscirà a schermo il tempo rimanente prima dello scadere del cronometro e poi il messaggio "Il bot nero sta pensando..." in attesa che calcoli la mossa e venga stampata a schermo. La partita continua nel loop fino al termine per scacco matto, stallo o fine tempo. Nel caso in cui venga selezionata la modalità [3] Umano vs Umano, la metodologia per eseguire le mosse e leggere il tempo restante è omologa al turno Umano della modalità [1] solo che saranno alternati tra turno bianco e nero per giocare in modalità locale.

7 Interpretazione dei risultati

La valutazione dei risultati avviene in maniera ottimale se viene utilizzata la modalità spettatore, poiché non occorre pensare anche alle mosse da giocare. Per l'esecuzione delle mosse, a seguito delle ottimizzazioni, si ottengono in apertura tempistiche risposta ampiamente sotto i 500ms. All'inizio del medio gioco la tempistica sale tra 1 e 3 s. Per tutto il resto della partita la tempistica di risposta oscilla tra i 5 e i 10 s con casistiche isolate sopra i 15 s in casi complessi o in endgame difficili. Data l'ottima efficienza della generazione delle mosse, ho provato a spingere il motore più in profondità (rispettivamente 6 in apertura, 7 in inizio medio gioco e 8 per mediogioco avanzato ed endgame). Le tempistiche di risposta si sono alzate, circa 1-3 s in apertura; tra i 5 e i 10 s in medio gioco e oltre i 12 s per l'endgame. Per la valutazione della precisione sono state effettuate simulazioni contro Stockfish: nel primo giro a piena potenza (ELO 3300 circa) il mio bot ha perso tutte le partite (40 su 40). Successivamente è stato abbassata la potenza a 1500 per una stima più equa. Le condizioni di gioco davano solamente 700 ms per la generazione della mosse e le simulazioni

sono state lanciate su circa 300 partite ottenendo un punteggio ELO di 1350 circa e posizionandosi nella fascia di giocatore medio/avanzato (i test sono stati eseguiti unicamente con la configurazione di profondità di 4 in apertura, 5 in inizio medio gioco, 6 in medio gioco inoltrato e 7 in endgame; dovendo simulare tante partite é stato impostato un tempo per mossa abbastanza ridotto).

8 Strategia dei Test

Sono stati preparati i test sulle singole parti del programma (generazione, struttura e bot) e i test sono stati mirati a testare quelle che sono le possibili criticità principali. Tutte le funzioni sono state testate anche se alcune in maniera indiretta con test congiunti che valutano più funzioni assieme. Molte funzioni lavorano accoppiate e testarle singolarmente porterebbe a una eccessiva complessità. Poiché è stato riorganizzato il codice migliorando l'incapsulamento, è risultato necessario implementare metodi pubblici con l'unico scopo di dare visibilità a metodi privati per poterli testare con doctest; tali metodi sono nominati `test_` e seguiti dal nome della funzione da testare.

9 Dichiarazione di utilizzo di AI

In conformità alle policy di ateneo dichiaro di aver fatto uso di Intelligenze Artificiali generative (Google Gemini e Claude) unicamente per

- l'analisi degli errori di compilazione
- ricerca di bug all'interno del codice (con scarsi risultati dato che spesso presentava problemi inesistenti in un codice diverso da quello fornito)
- consigli di ottimizzazione
- aiuto per la configurazione delle simulazione